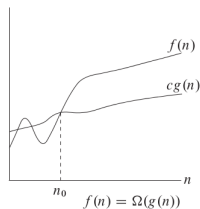
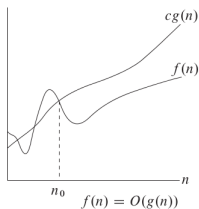
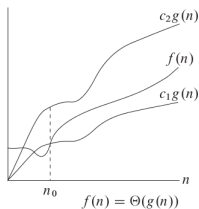


Complexity and Asymptotic Analysis

L.EIC

Algoritmos e Estruturas de Dados

2026/2027



AP Tomás, P Ribeiro

The Joy of Algorithms

“For me, **great algorithms are the poetry of computation**. Just like verse, they can be terse, allusive, dense and even mysterious. But **once unlocked, they cast a brilliant new light on some aspect of computing**.”

Francis Sullivan, The Joy of Algorithms, 2000

Algorithms + Data Structures = Program

A textbook by Niklaus Wirth, 1976

Algorithm: a well-defined computational procedure for solving a problem. It must **terminate after a finite number of steps**.

Correctness

It has to solve correctly **all instances** of the problem

Efficiency

The performance (**time** and **memory**) has to be adequate.

Efficient Algorithms

From textbook “Algorithms”, by Jeff Erickson, chapter 12.

<https://jeffe.cs.illinois.edu/teaching/algorithms/>

- A minimal requirement for an **algorithm** to be considered “efficient” is that its running time is bounded by a polynomial function of the input size: $O(n^c)$ for some constant c , where n is the size of the input.
(this kind of notation will be the focus of this class)
- Researchers recognized early on that not all problems can be solved this quickly, but had a hard time figuring out exactly which ones could and which ones couldn't.
- There are several so-called **NP-hard problems**, which most people believe cannot be solved in polynomial time, even though nobody can prove a super-polynomial lower bound.
(to be addressed later in L.EIC)

Some NP-hard Problems

To be addressed in Design of Algorithms (2nd semester)

- **SAT**: Given a CNF formula $\Phi(x_1, \dots, x_n) = C_1 \wedge \dots \wedge C_m$, is Φ satisfiable, i.e., is there a truth assignment that satisfies all clauses?
e.g., is $\Phi(p, q, r, s) = (\neg p \vee q \vee r) \wedge (\neg q \vee r \vee \neg s) \wedge (s \vee p) \wedge (\neg r \vee \neg q \vee p)$ satisfiable?
- **Partition**: Given a set $S = \{a_1, a_2, \dots, a_n\}$ of n positive integers, is there a set $A \subset S$ such that $\sum_{x \in A} x = \sum_{x \in S \setminus A} x$?
e.g., can we split $S = \{1, 4, 7, 15, 23, 42\}$ into two sets with the same sum?
- **Hamiltonian Cycle**: Given an undirected graph $G = (V, E)$, does G contain a cycle that visits all nodes exactly once?
- **TSP** (travelling salesperson problem): Given a **complete** weighted graph $G = (V, E, d)$, with $d(e) \in \mathbb{Z}^+$, for all $e \in E$, and $k \in \mathbb{Z}^+$, is there a hamiltonean cycle γ with $d(\gamma) \leq k$? Optimization version asks for **shortest hamiltonean cycle**.

Analysis of Algorithms

Complexity and Asymptotic Analysis

Brute force: For many non-trivial problems, there is a natural **brute force search algorithm** that checks every possible solution.

- Typically takes 2^n time or worse for inputs of size n .
- Unacceptable in practice.

Brute-force for SAT:

Given a CNF formula Φ in n (boolean) variables, enumerate all truth assignments to check whether any of them satisfies all clauses. In the worst case, 2^n **truth assignments** are checked.

Brute-force for Hamiltonian Cycle:

Given a graph $G = (V, E)$, with $|V| = n$, check whether any permutation of V defines a cycle in G . In the worst case, there are $n!$ **permutations** to check.

$n!$ is much worse than 2^n since
$$\lim_{n \rightarrow \infty} \frac{n!}{2^n} = \infty, \text{ that is } n! \gg 2^n$$

Asymptotic Growth

Common Functions

Function	Name	Examples
1	constant	summing two numbers
$\log n$	logarithmic	binary search, inserting in a heap
n	linear	find maximum value
$n \log n$	linearithmic	sorting (e.g.: mergesort, heapsort)
n^2	quadratic	verifying all pairs, bubblesort
n^3	cubic	Floyd-Warshall algorithm
2^n	exponential	exhaustive search (e.g.: subsets)
$n!$	factorial	list all permutations

Asymptotic Growth

A practical view

If an operation takes 10^{-9} seconds...

	$\log n$	n	$n \log n$	n^2	n^3	2^n	$n!$
10	< 0.01s	< 0.01s	< 0.01s	< 0.01s	< 0.01s	< 0.01s	< 0.01s
20	< 0.01s	< 0.01s	< 0.01s	< 0.01s	< 0.01s	< 0.01s	77 years
30	< 0.01s	< 0.01s	< 0.01s	< 0.01s	< 0.01s	1.07s	
40	< 0.01s	< 0.01s	< 0.01s	< 0.01s	< 0.01s	18.3 min	
50	< 0.01s	< 0.01s	< 0.01s	< 0.01s	< 0.01s	13 days	
100	< 0.01s	< 0.01s	< 0.01s	< 0.01s	< 0.01s	10^{13} years	
10^3	< 0.01s	< 0.01s	< 0.01s	< 0.01s	1s		
10^4	< 0.01s	< 0.01s	< 0.01s	0.1s	16.7 min		
10^5	< 0.01s	< 0.01s	< 0.01s	10s	11 days		
10^6	< 0.01s	< 0.01s	0.02s	16.7 min	31 years		
10^7	< 0.01s	0.01s	0.23s	1.16 days			
10^8	< 0.01s	0.1s	2.66s	115 days			
10^9	< 0.01s	1s	29.9s	31 years			

E.g.: for $n = 10^9$, the time to perform $\log_{10} n$ operations would be $10^{-9} \log_{10} 10^9 = 9 \times 10^{-9} \ll 0.01$ seconds

An Experience with Permutations

What is the running time of a program that goes through all permutations?

(the following times are approximated; what we want to point out is **order of growth**)

n = 6: 0.00009s

n = 7: 0.00014s

n = 8: 0.00048s

n = 9: 0.00324s

n = 10: 0.0309s

n = 11: 0.3263s

n = 12: 4.148s

n = 13: 51.156s

...

n = 20: aprox. 655 years!

How many permutations per second?

$$1.2 * 10^8$$

(about $8.5 * 10^{-9}$ seconds per permutation)

On Computer Speed

- Will a **faster computer** be of any help? **No!**

If $n = 20 \rightarrow 655$ years, hypothetically:

- ▶ 10x faster would still take 65.5 years
- ▶ 655x would still take 1 year
- ▶ 1 000 000x faster would still take about 5.7 hours, but
 $n = 21$ would take about 5 days
 $n = 22$ would take about 3.7 months

- The **growth rate** of the execution time is what matters!

Algorithmic performance vs Computer speed

A better algorithm on a slower computer **will always win** against a worst algorithm on a faster computer, for sufficiently large instances

Polynomial-time complexity

Desirable scaling property

When the input size doubles, the algorithm should only slow down by some constant factor C .

Polynomial time algorithm

There exists constants $a > 0$ and $b > 0$ such that on every input of size n , its running time is bounded by an^b primitive computational steps.

- If $T(n) = an$ then $T(2n)/T(n) = 2$
- If $T(n) = an^2$ then $T(2n)/T(n) = 2^2 = 4$
- If $T(n) = an^3$ then $T(2n)/T(n) = 2^3 = 8$
- If $T(n) = an^b$ then $T(2n)/T(n) = 2^b$

Many poly-time algorithms have both small constants and small exponents.

(This course unit will address algorithms of this kind mainly)

What can we do with execution time/space analysis?

Prediction

How much time/space does an algorithm need to solve a problem? How does it scale? Can we provide guarantees on its running time/memory?

Comparison

Is an algorithm A better than an algorithm B ? Fundamentally, what is the best we can possibly do on a certain problem?

- We will study a **methodology** to answer these questions
- We will focus mainly on execution time analysis

Random Access Machine (RAM)

- We need a **model** that is **generic** and **independent** from the language and the machine.
- We will consider a Random Access Machine (**RAM**)
 - ▶ Each **simple operation** (ex: $+$, $-$, $\leftarrow$, **If**) takes **1 step**
 - ▶ Loops and procedures, for example, are not simple instructions!
 - ▶ Each **access to memory** takes also **1 step**
- We can measure execution time by... **counting the number of steps as a function of the input size n : $T(n)$.**
- Operations are **simplified**, but this is useful
E.g.: summing two integers does not cost the same as dividing two reals, but we will see that on a global vision, these specific values are not important
- We can also define a **non-unitary cost model** and assign different constant costs to different elementary operations.

Asymptotic Notation – “Big-O notation”

Formalization (mathematical definition)

- $f(n) \in O(g(n))$ if there exist positive constants $n_0 \in \mathbb{Z}^+$ and $c \in \mathbb{R}^+$ such that $f(n) \leq c \times g(n)$ for all $n \geq n_0$.

$$f(n) \in O(g(n)) \Leftrightarrow \exists c \in \mathbb{R}^+ \exists n_0 \in \mathbb{Z}^+ \forall n \geq n_0 f(n) \leq cg(n)$$

- $f(n) \in \Omega(g(n))$ if there exist positive constants $n_0 \in \mathbb{Z}^+$ and $c \in \mathbb{R}^+$ such that $f(n) \geq c \times g(n)$ for all $n \geq n_0$.

$$f(n) \in \Omega(g(n)) \Leftrightarrow \exists c \in \mathbb{R}^+ \exists n_0 \in \mathbb{Z}^+ \forall n \geq n_0 f(n) \geq cg(n)$$

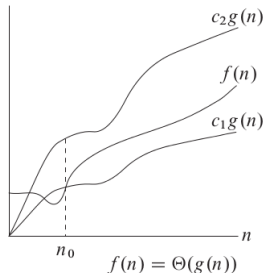
- $f(n) \in \Theta(g(n))$ if there exist positive constants $n_0 \in \mathbb{Z}^+$, $c_1 \in \mathbb{R}^+$ and $c_2 \in \mathbb{R}^+$ such that $c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$ for all $n \geq n_0$.

$$f(n) \in \Theta(g(n)) \Leftrightarrow \exists c_1, c_2 \in \mathbb{R}^+ \exists n_0 \in \mathbb{Z}^+ \forall n \geq n_0 c_1 g(n) \leq f(n) \leq c_2 g(n)$$

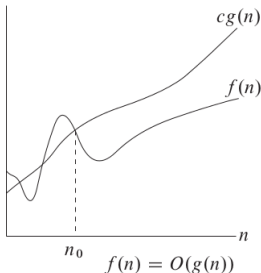
Asymptotic Notation

Graphical intuition

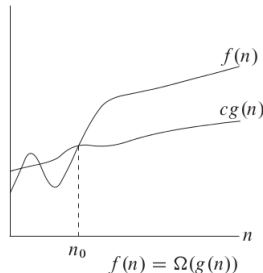
Θ



O



Ω



The definitions imply an n from which the function is bounded. The small values of n do not "matter".

Note: Some literature uses $=$ instead of $\in$

E.g.: $f(n) = O(g(n))$ means $f(n) \in O(g(n))$, actually.

Example: Time complexity of posMaxSorted

To find the position of the first occurrence of the maximum value of v , assuming that $v[0] \leq v[1] \leq \dots \leq v[n-1]$.

```
int posMaxSorted(int v[], int n) {  
    int i = n-2;  
    while (i >= 0 && v[i] == v[i+1]) i--;  
    return i+1;  
}
```

Counting instructions:	best	worst
int i = n-2	1	1
Test i >= 0	1	n
v[i] == v[i+1]	1	$n-1$
i--	0	$n-1$
return i+1	1	1
	4	$3n$

Worst case (all equal): $\Theta(n)$ time

Best case ($v[n-2] < v[n-1]$): $\Theta(1)$

Conclusion (what can we say for a generic instance?): takes $\mathbf{O}(n)$ time.

Example: Time complexity of posMaxSorted (cont.)

non-unitary cost model

```
int posMaxSorted(int v[], int n) {  
    int i = n-2;  
    while (i >= 0 && v[i] == v[i+1]) i--;  
    return i+1;  
}
```

Assigning different **constant costs** to different **elementary operations**...

	best	worst	
int i = n-2	c_1	c_1	$T_{best}(n) = c_1 + c_2 + c_3 + c_5$
Test i >= 0	c_2	$c_2 n$	$T_{worst}(n) = c_1 + c_2 n + (c_3 + c_4)(n - 1) + c_5$
v[i] == v[i+1]	c_3	$c_3(n - 1)$	
i--	0	$c_4(n - 1)$	Therefore, there is a constant $C \in \mathbb{R}^+$ such
return i+1	c_5	c_5	that, for all $n \geq 1$, it holds $T(n) \leq Cn$, for
			every instance, that is, $T(n) \in \mathcal{O}(n)$.

We can take $C = c_1 + c_2 + c_3 + c_4 + c_5$ because

$$T(n) \leq T_{worst}(n) = c_1 + c_2 n + (c_3 + c_4)(n - 1) + c_5 \leq (c_1 + c_2 + c_3 + c_4 + c_5)n$$

Example: Time complexity of posMaxSorted

To find the position of the first occurrence of the maximum value of v , assuming that $v[0] \leq v[1] \leq v[2] \leq \dots \leq v[n-1]$.

```
int posMaxSorted(int v[], int n) {  
    int i = n-2;  
    while (i >= 0 && v[i] == v[i+1])  
        i--;  
    return i+1;  
}
```

Best case: The maximum occurs only once. Time complexity: $\Theta(1)$

Worst case: All elements are equal. Time complexity: $\Theta(n)$

Conclusion: the time complexity of $\text{posMaxSorted}(v, n)$ is $O(n)$.

Questions? Is there a more efficient algorithm for the problem, e.g., whose running time is $O(\log n)$? (Yes. Based on binary search!)

posMaxSorted in $O(\log_2 n)$ based on binary search

To find the position of the first occurrence of the maximum value of v , assuming that $v[0] \leq v[1] \leq \dots \leq v[n-1]$, with $n \geq 1$.

Improved version based on binary-search:

```
int posMaxSorted(int v[], int n) {
    int low = 0, high = n-1, middle, max = v[n-1];
    while (low < high) {
        middle = low + (high-low)/2;    // avoids overflow
        if (v[middle] == max) high = middle;
        else low = middle + 1;
    }
    return low;
}
```

Loop invariant: In all tests of the while condition, it holds $low \leq high$ and $v[high]=v[n-1]$. If $low < high$ then $middle = low+(high-low)/2 < high$.

Progress: $high-low$ decreases in every iteration

Termination Loop ends with $high=low$. So, **return low** gives correct value.

Example: Evaluate a polynomial at a point

Problem: Compute the value of $p(x) = \sum_{k=0}^n a_k x^k$ at a point x , given the array of coefficients, the value of x and the degree n of the polynomial.

Consider the two functions in a pseudocode

```
eval_pol1(a,x,n):  
    res = a[0]  
    for k = 1 to n do  
        pot = x  
        for j = 2 to k do  
            pot = pot*x  
        res = res + a[k]*pot  
    return res
```

```
eval_pol2(a,x,n):  
    res = a[0]  
    pot = x  
    for k = 1 to n do  
        res = res + a[k]*pot  
        pot = pot*x  
    return res
```

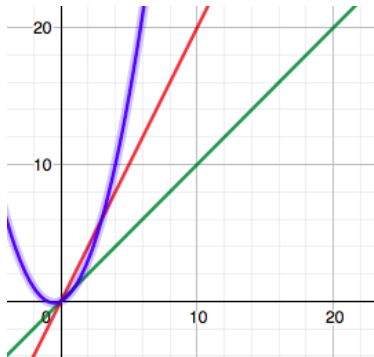
Time complexity: The number of computation steps is very different in the two functions, e.g., we have $\sum_{k=1}^n (1 + (k-1)) = n(n+1)/2$ products in `eval_pol1(a,x,n)` and just $2n$ in `eval_pol2(a,x,n)`.

$n(n+1)/2 \gg 2n$ if n is large

$\Theta(n^2)$ instead of $\Theta(n)$

Comparing

n	2n	$n(n+1)/2$
1	2	1
2	4	3
3	6	6
4	8	10
5	10	15
6	12	21
7	14	28
⋮	⋮	⋮
50	100	1275
100	200	5050
200	400	20100
400	800	80200
1000	2000	500500



Horner's method: Performs only n products to compute $p(x) = \sum_{k=0}^n a_k x^k$.

Example: evaluate a polynomial by Horner's method

Horner's method

Performs only n products to compute $p(x) = \sum_{k=0}^n a_k x^k$, given x .

```
eval_pol_Horner(a,x,n):  
    res = a[n]  
    for k = n-1 to 0 with step -1 do  
        res = res*x + a[k]  
    return res
```

Also $\Theta(n)$ but with only n products, instead $2n$

Idea: Computes $p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_2 x^2 + a_1 x + a_0$ incrementally, starting from a_n .

$$\begin{aligned} & a_n \\ & a_n x + a_{n-1} \\ & (a_n x + a_{n-1})x + a_{n-2} \\ & \vdots \\ & (((((((a_n x + a_{n-1})x + a_{n-2})x + a_{n-3})x + \dots)x + a_3)x + a_2)x + a_1)x + a_0 = p(x) \end{aligned}$$

Asymptotic Notation

Defining bounds for order of growth

$$f(n) \in O(g(n))$$

It means that $c \times g(n)$ is an **upper bound** of $f(n)$, for **all** $n \geq n_0$

$$f(n) \in \Omega(g(n))$$

It means that $c \times g(n)$ is a **lower bound** of $f(n)$, for **all** $n \geq n_0$

$$f(n) \in \Theta(g(n))$$

It means $c_1 \times g(n)$ is a **lower bound** of $f(n)$ and $c_2 \times g(n)$ is an **upper bound** of $f(n)$, for **all** $n \geq n_0$

where $c, c_1, c_2 \in \mathbb{R}^+$ and $n_0 \in \mathbb{Z}_0^+$ are **constants** chosen appropriately

Asymptotic Notation

Defining orders of growth

$$f(n) \in O(g(n)) \Leftrightarrow \exists c \in \mathbb{R}^+ \exists n_0 \in \mathbb{Z}^+ \forall n \geq n_0 f(n) \leq cg(n)$$

$$f(n) \in \Omega(g(n)) \Leftrightarrow \exists c \in \mathbb{R}^+ \exists n_0 \in \mathbb{Z}^+ \forall n \geq n_0 f(n) \geq cg(n)$$

$$f(n) \in \Theta(g(n)) \Leftrightarrow \exists c_1, c_2 \in \mathbb{R}^+ \exists n_0 \in \mathbb{Z}^+ \forall n \geq n_0 c_1g(n) \leq f(n) \leq c_2g(n)$$

$O(g(n))$, $\Omega(g(n))$ and $\Theta(g(n))$ denote **sets of functions in natural numbers** that consist of all functions $f : \mathbb{N} \rightarrow \mathbb{R}_0^+$ related to the function $g : \mathbb{N} \rightarrow \mathbb{R}_0^+$ by the corresponding condition.

Remark (abuse of notation): in these definitions $f(n)$ and $g(n)$ are used with two meanings: $f(n) \leq cg(n)$ refers to the images of n by f and by g , whereas in $f(n) \in O(g(n))$, we refer to the functions $f : \mathbb{N} \rightarrow \mathbb{R}^+$ and $g : \mathbb{N} \rightarrow \mathbb{R}^+$.

Types of algorithm analysis

Worst Case analysis: (the most common)

- $T(n)$ = maximum amount of time for any input of size n

Average Case analysis: (used sometimes)

- $T(n)$ = average time on all inputs of size n
- Implies knowing the **statistical distribution** of the inputs

Best Case analysis: ("deceiving")

- It's almost like "cheating" with an algorithm that is fast just for **some** of the inputs

Asymptotic Notation

Using the mathematical definition

- $99n^2 \in \Theta(n^2)$
 - ▶ $n^2 \leq 99n^2 \leq 99n^2$, for all $n \geq 1$.
 - ▶ Therefore, there exist $c_1, c_2 \in \mathbb{R}^+$ and $n_0 \in \mathbb{Z}^+$ such that $c_1 n^2 \leq 99n^2 \leq c_2 n^2$, for all $n \geq n_0$.
 - ▶ E.g.: we can take $c_1 = 1$, $c_2 = 99$ and $n_0 = 1$.
- $3n^3 - 5n^2 + 100 \in \Theta(n^3)$ because
 - ▶ $3n^3 - 5n^2 + 100 \geq 2n^3$, for all $n \geq 5$, since $n^3 - 5n^2 \geq 0$ for all $n \geq 5$
 - ▶ $3n^3 - 5n^2 + 100 \leq 3n^3 + 5n^2 + 100 \leq 3n^3 + 5n^3 + 100n^3 = 108n^3$, for all $n \geq 1$.
 - ▶ Therefore, there exist $c_1, c_2 \in \mathbb{R}^+$ and $n_0 \in \mathbb{Z}^+$ such that $c_1 n^3 \leq 3n^3 - 5n^2 + 100 \leq c_2 n^3$, for all $n \geq n_0$.
 - ▶ E.g.: we can take $c_1 = 2$, $c_2 = 108$ and $n_0 = 5$.

Asymptotic Notation

A few consequences

Recall:

$$f(n) \in O(g(n)) \Leftrightarrow \exists c \in \mathbb{R}^+ \exists n_0 \in \mathbb{Z}^+ \forall n \geq n_0 f(n) \leq cg(n)$$

$$f(n) \in \Omega(g(n)) \Leftrightarrow \exists c \in \mathbb{R}^+ \exists n_0 \in \mathbb{Z}^+ \forall n \geq n_0 f(n) \geq cg(n)$$

$$f(n) \in \Theta(g(n)) \Leftrightarrow \exists c_1, c_2 \in \mathbb{R}^+ \exists n_0 \in \mathbb{Z}^+ \forall n \geq n_0 c_1g(n) \leq f(n) \leq c_2g(n)$$

A few consequences:

- $f(n) \in \Theta(g(n)) \Leftrightarrow f(n) \in O(g(n))$ and $f(n) \in \Omega(g(n))$
- $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$
- $f(n) \in O(g(n)) \Leftrightarrow g(n) \in \Omega(f(n))$

Asymptotic Notation

A few practical rules (NOT MATHEMATICAL DEFINITIONS!)

- **Multiplying by a constant** does not affect the behavior:
 $c \times f(n) \in \Theta(f(n))$, for $c \in \mathbb{R}^+$
 $99 \times n^2 \in \Theta(n^2)$
 $\Theta(500n^2) = \Theta(n^2) = \Theta(1/10000n^2)$
- On a polynomial of the form $a_x n^x + a_{x-1} n^{x-1} + \dots + a_2 n^2 + a_1 n + a_0$ we can focus on the term with the **largest exponent**:
 $3n^3 - 5n^2 + 100 \in \Theta(n^3)$
 $0.6n^4 + 20^2 \in \Theta(n^4)$
- On a sum/subtraction we can focus on the **dominant** term:
 $2^n + 6n^3 \in \Theta(2^n)$
 $n! - 3n^2 \in \Theta(n!)$

WHY ARE THESE PRACTICAL RULES VALID?

Try to prove each one using the **mathematical definitions**

Examples

- $\log_2(n) \in O(n)$ because $\log_2 n \leq n$, for all $n \geq 1$.

Therefore, we can take $c = 1$ and $n_0 = 1$ in the definition.

- $\log_2(n) \notin \Omega(n)$ because $\lim_{n \rightarrow \infty} \frac{\log_2 n}{n} = 0$.

$\lim_{n \rightarrow \infty} \frac{\log_2 n}{n} = 0$ means that $\forall \varepsilon \in \mathbb{R}^+ \exists n'_0 \in \mathbb{Z}^+ \forall n \geq n'_0 \quad \frac{\log_2 n}{n} < \varepsilon$.

This condition implies that $\forall c \in \mathbb{R}^+ \forall n_0 \in \mathbb{Z}^+ \exists n \geq n_0 \quad \log_2 n < cn$, which is $\log_2(n) \notin \Omega(n)$.

- $O(n) \subseteq O(n^2)$ since for all $f(n) \in O(n)$, we have $f(n) \in O(n^2)$.

By definition, $f(n) \in O(n)$ iff $\exists c \in \mathbb{R}^+ \exists n_0 \in \mathbb{Z}^+ \forall n \geq n_0 \quad f(n) \leq cn$.

If $f(n) \in O(n)$, then it is true that $\exists c' \in \mathbb{R}^+ \exists n'_0 \in \mathbb{Z}^+ \forall n \geq n'_0 \quad f(n) \leq cn'$.

We can take $c' = c$ and $n'_0 = n$, since $cn \leq cn^2$.

Questions?

- $O(n) \subset O(n^2)$ Yes
- $\sqrt{n} \in \Omega(n)$ No
- $\Omega(n \log_2 n) \subset \Omega(n)$ Yes
- $\Theta(\log_a n) = \Theta(\log_b n)$, para $a, b \in \mathbb{R}^+$, $a \neq b$, $a, b > 1$? Yes
- $O(2^n) = O(3^n)$ No
- $O(2^n) \subset O(3^n)$ Yes
- $\Theta(2n) = \Theta(3n)$ Yes
- $\mathbf{f}(n) \in \Omega(1)$, for all $\mathbf{f} : \mathbb{N} \rightarrow \mathbb{R}^+$? (Therefore, $O(1) = \Theta(1)$) Yes

Exercise

Consider the following function in C/C++.

```
int func(int x[], int n) {  
    int c = 0;  
    for(int i = 0; i < n-1; i += 2)  
        if (x[i] > x[i+1]) { x[i] -= x[i+1]; c++; }  
    return c;  
}
```

- 1 State the problem `int func(int x[], int n)` solves.
- 2 Give a formal proof that the function is correct. Start by writing the loop invariant, assuming that $x[j] = a_j$, for $0 \leq j \leq n-1$, initially.
- 3 How many times is each instruction and each test executed in the worst case and in the best case? Describe a best case instance and a worst case instance of size n .
- 4 Characterize the asymptotic time complexity of this function.

Asymptotic Notation

Dominance

When is a function **better** than another?

- If we want to minimize time, "**smaller**" functions are better
- A function **dominates** another one if as n grows it keeps getting infinitely larger
- Mathematically: $f(n) \gg g(n)$ if $\lim_{n \rightarrow \infty} g(n)/f(n) = 0$

Dominance Relations

$$n! \gg 2^n \gg n^3 \gg n^2 \gg n \log n \gg n \gg \sqrt{n} \gg \log n \gg 1$$

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$$

$$\Omega(1) \supset \Omega(\log n) \supset \Omega(n) \supset \Omega(n \log n) \supset \Omega(n^2) \supset \Omega(n^3) \supset \Omega(2^n) \supset \Omega(n!)$$

Predicting the execution time

Pre-requirements:

- An implementation with time complexity given by $f(n)$
- A (small) test case with input of size n_1
- The running time of the program on that input: $time(n_1)$

We want to **estimate** the running time for a (similar) input of size n_2 .

How to do it?

Estimating the execution time

$f(n_2)/f(n_1)$ is the growth rate of the function (from n_1 to n_2)

$$time(n_2) \approx f(n_2)/f(n_1) \times time(n_1)$$

(here, $\approx$ means “approximately equal to”; we can use $=$ instead)

Predicting the execution time

An example

- Imagine a program with time complexity $\Theta(n^2)$ that takes **1 second** for an input of size **5 000**. What is my **estimation** for the execution time for an input of size **10 000**?

$$f(n) = n^2$$

$$n_1 = 5\,000$$

$$\text{time}(n_1) = 1$$

$$n_2 = 10\,000$$

$$\begin{aligned}\text{time}(n_2) &= f(n_2)/f(n_1) \times \text{time}(n_1) = \\ &= 10\,000^2/5\,000^2 \times 1 = 4 \text{ seconds}\end{aligned}$$

Predicting the execution time

About the growth rate

Let's see what happens when we **double the input** for some of the more common functions (independently of the machine used!):

$$\text{time}(2n) = f(2n)/f(n) \times \text{time}(n)$$

- n : $2n/n = 2$. The time is the **double**!
- n^2 : $(2n)^2/n^2 = 4n^2/n^2 = 4$. Time increases **4x**!
- n^3 : $(2n)^3/n^3 = 8n^3/n^3 = 8$. Time increases **8x**!
- 2^n : $2^{2n}/2^n = 2^{2n-n} = 2^n$. Time grows **2^n times**!
Example: If $n = 5$, the time for $n = 10$ will be **32x** more!
Example: If $n = 10$, the time for $n = 20$ will be **1024x** more!
- $\log_2(n)$: $\log_2(2n)/\log_2(n)$. It increases $\frac{\log_2(2n)}{\log_2(n)}$ vezes!
Example: If $n = 5$, the time for $n = 10$ will be **1.43x** more!
Example: If $n = 10$, the time for $n = 20$ will be **1.3x** more!

Analyzing the complexity of programs

Let's see more concrete examples:

- **Case 1 Loops** (and summations)
- **Case 2 Recursive Functions** (and recurrences)

Recall: Loops and Summations

Some relevant cases for counting

```
int count = 0;
for (int i=0; i<1000; i++)
    for (int j=i; j<1000; j++)
        count++;
cout << count << endl;
```

(its time complexity is $O(1)$ because even 1000×1000 is a constant)

What does this program write?

The value of $1000 + 999 + 998 + 997 + \dots + 2 + 1$

```
// a better  $O(1)$  solution - using math background
cout << (1000+1)*1000/2 << endl;
```

Recall: Loops and Summations

Arithmetic progression: a sequence of numbers such that the difference d between any two consecutive terms is constant. Let a_1 be the first term.

- 1, 2, 3, 4, 5, ($d = 1, a_1 = 1$)
- 3, 5, 7, 9, 11, ($d = 2, a_1 = 3$)

How to calculate sums of consecutive terms in an arithmetic progression?

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 = (1 + 8) + (2 + 7) + (3 + 6) + (4 + 5) = 4 \times 9$$

Expression defining the sum from a_p to a_q

$$S(p, q) = \sum_{i=p}^q a_i = \frac{(q-p+1)(a_p+a_q)}{2}$$

Expression defining the sum of the first n terms

$$S_n = \sum_{i=1}^n a_i = \frac{n(a_1+a_n)}{2}$$

Recall: Loops and Summations

```
int count = 0;
for (int i=0; i<1000; i++)
    for (int j=i; j<1000; j++)
        count++;
cout << count << endl;
```

What does this program write?

$1000 + 999 + 998 + 997 + \dots + 2 + 1$

It writes $S_{1000} = \frac{1000 \times (1000+1)}{2} = 500500$

Recall: Loops and Summations

```
int count = 0;
for (int i=0; i<n; i++)
    for (int j=i; j<n; j++)
        count++;
cout << count << endl;
```

What is the execution time?

It is going to execute S_n increments:

$$S_n = \sum_{i=1}^n a_i = \frac{n \times (1+n)}{2} = \frac{n+n^2}{2} = \frac{1}{2}n^2 + \frac{1}{2}n.$$

It executes $\Theta(n^2)$ steps

Recall: Loops and Summations

If you want to know more about interesting summations on this context, take a look at *Appendix A* of the *Introduction to Algorithms* book.

Note that c cycles do not imply $\Theta(n^c)$!

```
for (int i=0; i<n; i++)  
    for (int j=1; j<5; j++)
```

$\Theta(n)$ (Two loops but not $\Theta(n^2)$)

```
for (int i=1; i<=n; i++)  
    for (int j=1; j<=i*i; j++)
```

$\Theta(n^3)$ $1^2 + 2^2 + 3^2 + \dots + n^2 = \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$

```
i = n;  
while (i>0) i = i/2;
```

$\Theta(\log n)$ each time i becomes reduced to a half ($n/2^p \leq 1$ iff $p \geq \log_2 n$)

Example: Time complexity of Posmax

First occurrence of the maximum of $v[k], v[k+1], \dots, v[n]$

Write a function `POSMAX( $v, k, n$ )` that returns the index of the first occurrence of the maximum element in the segment $v[k], v[k+1], \dots, v[n]$ of the array v . Assume that $k \leq n$ and that the segment is within v .

Example: Time complexity of Posmax

In a pseudocode

(here, $\leftarrow$ means assignment)

POSMAX(v, k, n)

1. $pmax \leftarrow k$;
2. $i \leftarrow k + 1$;
3. while $i \leq n$ do
4. if $v[i] > v[pmax]$ then
5. $pmax \leftarrow i$;
6. $i \leftarrow i + 1$;
7. return $pmax$;

```
// in C++
int Posmax(int v[], int k, int n) {
    int pmax = k;
    int i = k+1;
    while (i <= n) {
        if (v[i] > v[pmax])
            pmax = i;
        i++;
    }
    return pmax;
}
```

Example: Time complexity of Posmax

POSMAX(v, k, n)

Running times (cost model)

```
1.   $pmax \leftarrow k$ ;  
2.   $i \leftarrow k + 1$ ;  
3.  while  $i \leq n$  do  
4.    if  $v[i] > v[pmax]$  then  
5.       $pmax \leftarrow i$ ;  
6.       $i \leftarrow i + 1$ ;  
7.  return  $pmax$ ;
```

```
 $c_1$ : assign value of a variable to another one  
 $c_2$ : assign value of expression to a variable  
 $c_3$ : test the condition and transfer control  
 $c_4$ : access, test and transfer control  
 $c_1$   
 $c_6$ : increment the value of a variable  
 $c_7$ : return from the function with a value
```

- Executes instructions 1, 2 e 7 a single time. Executes the test in line 3 for $i = k + 1, \dots, n, n + 1$, that is, $(n + 1) - (k + 1) + 1 = n - k + 1$ times.
- Executes line 4 for $i = k + 1, \dots, n$, that is $n - (k + 1) + 1 = n - k$ times. Similar to instruction 6.
- **What about for instruction 5?** It depends on the instance.
 - ▶ **best case:** does not execute instr. 5. It happens when $v[k]$ is strictly greater than the remaining elements.
 - ▶ **worst case:** $v[k] < v[k + 1] < \dots < v[n - 1] < v[n]$ executes $n - k$ times.

Example: Time complexity of Posmax

Posmax(v, k, n)

```
1.   $pmax \leftarrow k;$ 
2.   $i \leftarrow k + 1;$ 
3.  while  $i \leq n$  do
4.    if  $v[i] > v[pmax]$  then
5.       $pmax \leftarrow i;$ 
6.       $i \leftarrow i + 1;$ 
7.  return  $pmax;$ 
```

Running times (cost model)

c_1 : assign value of a variable to another one

c_2 : assign value of expression to a variable

c_3 : test the condition and transfer control

c_4 : access, test and transfer control

c_1

c_6 : increment the value of a variable

c_7 : return from the function with a value

• Best case: $T(n, k) = c_1 + c_2 + c_7 + (n - k + 1)c_3 + (n - k)(c_4 + c_6)$.

• Worst case: $T(n, k) = c_1 + c_2 + c_7 + (n - k + 1)c_3 + (n - k)(c_4 + c_6 + c_1)$.

For all instances, it holds

$$\underbrace{c_1 + c_2 + c_7 + c_3 + (c_3 + c_4 + c_6)(n - k)}_{\text{best case}} \leq T(n, k) \leq \underbrace{c_1 + c_2 + c_7 + c_3 + (c_3 + c_4 + c_6 + c_1)(n - k)}_{\text{worst case}}$$

Example: Time complexity of Posmax

For all instances, it holds

$$\underbrace{c_1 + c_2 + c_7 + c_3 + (c_3 + c_4 + c_6)(n - k)}_{\text{best case}} \leq T(n, k) \leq \underbrace{c_1 + c_2 + c_7 + c_3 + (c_3 + c_4 + c_6 + c_1)(n - k)}_{\text{worst case}}$$

if we define $a = \min_t c_t$ and $b = \max_t c_t$, we conclude that

$$4a + 3a(n - k) \leq T(n, k) \leq 4b + 4b(n - k),$$

and

$$3a(n - k + 1) \leq 4a + 3a(n - k) \leq T(n, k) \leq 4b + 4b(n - k) = 4b(n - k + 1).$$

There exist constants $c', c'' \in \mathbb{R}^+$ such that $c'(n - k + 1) \leq T(n, k) \leq c''(n - k + 1)$, namely $c' = 3a$ and $c'' = 4b$.

This means that $T(n, k) \in \Theta(n - k + 1)$, that is $T(n, k)$ is **linear** in the length of the segment $[k, n]$ of v that will be analysed when we call $\text{Posmax}(v, k, n)$.

Example: Time complexity of Posmax

Another way of finding suitable constants c' and c'' :

For all instances, it holds

$$\underbrace{c_1 + c_2 + c_7 + c_3 + (c_3 + c_4 + c_6)(n - k)}_{\text{best case}} \leq T(n, k) \leq \underbrace{c_1 + c_2 + c_7 + c_3 + (c_3 + c_4 + c_6 + c_1)(n - k)}_{\text{worst case}}$$

Therefore, since $n \geq k$ and all constants are positive, we have

$$T(n, k) \geq c_3 + c_3(n - k) = c_3(n - k + 1)$$

and

$$T(n, k) \leq (c_2 + c_7 + c_3 + c_4 + c_6 + c_1)(n - k + 1)$$

meaning that, we could take $c' = c_3$ and $c'' = c_2 + c_7 + c_3 + c_4 + c_6 + c_1$ to prove that there exist $c', c'' \in \mathbb{R}^+$ such that $c'(n - k + 1) \leq T(n, k) \leq c''(n - k + 1)$ and conclude that $T(n, k) \in \Theta(n - k + 1)$.

Example: Time complexity of Posmax

Linear in the number of elements in the segment

Let $N = n - k + 1$ be the **number of elements in the segment** $[k, n]$ of v that will be analysed when we call $\text{Posmax}(v, k, n)$. We saw that

- there exists $c' \in \mathbb{R}^+$ such that $T(N) \geq c'N$, for all $N \geq 1$, i.e., $T(N) \in \Omega(N)$.

$T(N)$ is **at least linear in N**

- there exists $c'' \in \mathbb{R}^+$ such that $T(N) \leq c''N$, for all $N \geq 1$, i.e., $T(N) \in O(N)$.

$T(N)$ is **at most linear in N**

- there exist $c', c'' \in \mathbb{R}^+$ such that $c'N \leq T(N) \leq c''N$, for all $N \geq 1$, i.e., $T(N) \in \Theta(N)$.

$T(N)$ is **exactly linear in N**

By definition of $\Theta(g(n))$, $\Omega(g(n))$ and $O(g(n))$:

$$\Theta(g(n)) = \Omega(g(n)) \cap O(g(n))$$

Correctness and complexity of recursive programs

For recursive functions:

- Correctness proofs often based on **strong induction** or **structural induction**
- Complexity analysis often requires **solving a recurrence**

MergeSort and QuickSort

Based on divide and conquer

- Algorithms that are expressed in a **recursive** way
- Follow the **divide and conquer** strategy:

Divide and Conquer

Divide the problem into a set of subproblems which are smaller instances of the same problem

Conquer the subproblems solving them recursively. If the problem is small enough, solve it directly.

Combine the solutions of the smaller subproblems on a solution for the original problem

MergeSort

Divide: partition the initial array in two halves

Conquer: recursively sort each half. If we only have one item, it is sorted.

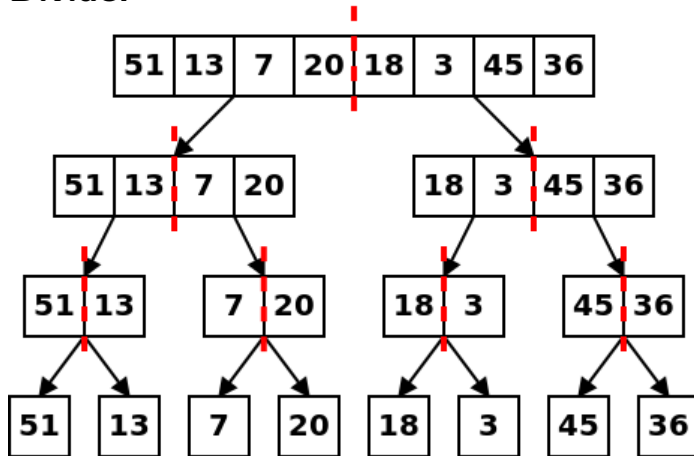
Combine: merge the two sorted halves in a final sorted array

Sorting algorithms and recurrences will be covered in the next lecture

Divide and Conquer

MergeSort

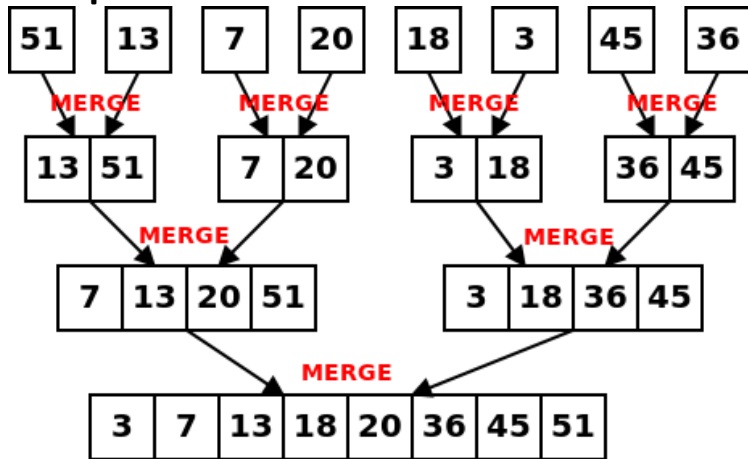
Divide:



Divide and Conquer

MergeSort

Conquer:



MergeSort

The recurrence that defines its runtime

- **D(n)** - Time to partition an array of size n in two halves
- **M(n)** - Time to merge two sorted arrays of size n
- **T(n)** - Time for a MergeSort on an array of size n

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ D(n) + 2T(n/2) + M(n) & \text{if } n > 1 \end{cases}$$

becomes

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

In practice, we are ignoring certain details, but it suffices

(ex: when n is odd, the size of subproblem is not exactly $n/2$)

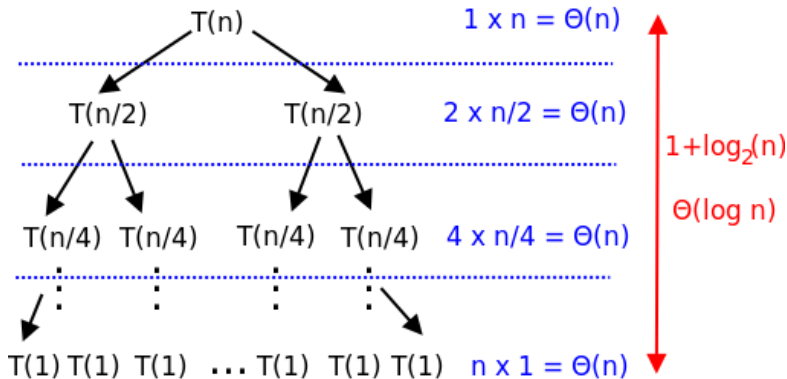
How to solve this recurrence?

(for a cleaner explanation we will assume $n = 2^k$, but the results holds for any n)

MergeSort

Recursion Tree

Let's draw the **recursion tree**:



Worst case: adding everything we get that **MergeSort** is $\Theta(n \log_2 n)$

Divide and Conquer

Maximum by Divide and Conquer

A recursive algorithm is not always **linearithmic**, i.e, $O(n \log n)$

Let's see another example. Imagine that you want to compute the **maximum** of an array of size n .

A simple **linear search** would be enough, but let's design a divide and conquer algorithm.

Computing the maximum

Divide: partition the initial array in two halves

Conquer: recursively compute the maximum in each half. If we only have one number, it is the maximum

Combine: compare the maximum of each half and keep the largest one

Divide and Conquer

Maximum by Divide and Conquer

```
// maximum value in the segment v[low],v[low+1],...,v[high]
// assume 0 <= low <= high < v.size()
// e.g. call: max_DC(v,0,v.size()-1)
```

```
int max_DC(vector<int> &v, int low, int high) {
    if (low == high) return v[low];

    int middle = low + (high-low)/2;

    int max1 = max_DC(v,low,middle);
    int max2 = max_DC(v,middle+1,high);

    if (max1 >= max2) return max1;
    return max2;
}
```

Termination: remark that if $low < high$ then $low \leq middle < high$

Proof of correctness: use **strong induction** on the size $high-low+1$

Recall Strong Induction

To **prove** $\forall n \geq n_0 \ P(n)$, where $P(n)$ is some property, by **mathematical induction**, using the **strong version of the principle** we can show that

- **Base Case:**

$P(n_0)$ is true.

- **Induction Step (uses a stronger hypothesis):**

For all $k \geq n_0$, **if** $P(j)$ holds for all j such that $n_0 \leq j \leq k$ **then** $P(k+1)$ holds.

In the **weak version**, the induction step is:

For all $k \geq n_0$, **if** $P(k)$ holds **then** $P(k+1)$ holds.

Divide and Conquer

Maximum by Divide and Conquer

What is the **execution time** of this algorithm?

To simplify, let's again admit that n is a power of 2.
(the results are similar in their essence for other cases)

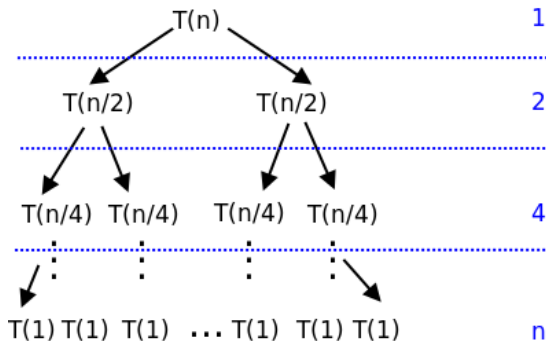
$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2T(n/2) + \Theta(1) & \text{se } n > 1 \end{cases}$$

How does this differ from the MergeSort recurrence?

How to **solve** it?

Divide and Conquer

Maximum by Divide and Conquer



In total we spend $1 + 2 + 4 + \dots + n = \sum_{i=0}^{\log_2(n)} 2^i = 2n - 1$

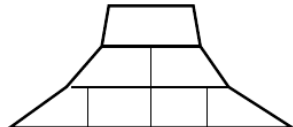
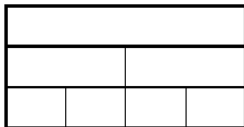
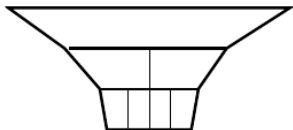
What dominates the sum? Note that $2^k = 1 + \sum_{i=0}^{k-1} 2^i$.

The last level dominates the weight and thus the algorithm is $\Theta(n)$

Time complexity: recursive algorithms

The most common recursive algorithms fall on **one of three cases**:

- The time is (uniformly) **distributed** along the recursion tree (e.g. mergesort)
- The time is dominated by the **last level** of the recursion (e.g. maximum by divide and conquer)
- The time is dominated by the **top level** of the recursion (e.g. naive matrix multiplication using divide and conquer)



Recurrences

Notation

It is common to assume that $T(1) = \Theta(1)$. In these cases we can simply write $T(n)$ to describe a recurrence.

- **MergeSort:** $T(n) = 2T(n/2) + \Theta(n)$
- **Maximum by D&C:** $T(n) = 2T(n/2) + \Theta(1)$

Divide and Conquer

More recurrences

Sometimes we have an algorithm that reduces the problem to a single subproblem.

In this case we can say we use **decrease and conquer**

- **Binary Search:**

On a sorted array of size n , compare with the middle element and continue the search on one half

$$T(n) = T(n/2) + \Theta(1) \qquad \Theta(\log n)$$

- **Max with "tail recursion":** On an array of size n , recursively find the maxim of the entire array except the first element and then compare with that first element

$$T(n) = T(n-1) + \Theta(1) \qquad \Theta(n)$$

Solving Recurrences: The Master Theorem

Out of the scope of AED

The **Master Theorem** provides solutions for more general recurrences:

Master Theorem

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence $T(n) = aT(n/b) + f(n)$, where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ has the following asymptotic bounds:

- 1 If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
- 2 If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log_2 n)$.
- 3 If $f(n) = \Omega(n^{\log_b a + \varepsilon})$, for some constant $\varepsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$.

E.g.: By Case 3, $T(n) = 2T(n/2) + n^2$ solves to $T(n) \in \Theta(n^2)$, since $a = b = 2$, $\log_b a = 1$, $f(n) = n^2 \in \Omega(n^{\log_b a + \varepsilon}) = \Omega(n^{1+\varepsilon})$, for a constant $\varepsilon > 0$ (for instance, $\varepsilon = 1$), $2f(n/2) = 2(n/2)^2 = n^2/2 \leq cf(n)$, for $c = 1/2 < 1$,